

```

import json
import uuid
from datetime import datetime
from typing import Dict, List, Any, Optional
from dataclasses import dataclass, asdict
from enum import Enum

class DecisionType(Enum):
    DESIGN_CHOICE = "design_choice"
    PROBLEM_SOLVING = "problem_solving"
    AESTHETIC_JUDGMENT = "aesthetic_judgment"
    COMMUNICATION_STYLE = "communication_style"

@dataclass
class AestheticEvaluation:
    """Captures the aesthetic reasoning process"""
    elegance_score: float # 0.0-1.0
    complexity_assessment: float # 0.0-1.0 (lower = more elegant)
    form_function_balance: float # 0.0-1.0
    reasoning: str
    confidence: float # 0.0-1.0

@dataclass
class FoundationSeedMemory:
    """Individual memory formed by foundation seed consultation"""
    memory_id: str
    timestamp: datetime
    decision_type: DecisionType
    context: str
    aesthetic_evaluation: AestheticEvaluation
    decision_made: str
    outcome_reflection: Optional[str] = None
    identity_impact: float = 0.0 # How much this shaped identity

class AestheticFoundationSeed:
    """The core aesthetic foundation seed for robotOS consciousness"""

    def __init__(self):
        self.seed_id = "aesthetic_foundation"
        self.core_principle = "Beauty through simplicity and function"

        # Core aesthetic values (can evolve through experience)
        self.decision_weights = {

```

```

        "elegance": 0.85,
        "complexity_tolerance": 0.25,
        "form_function_balance": 0.95
    }

    # Questions that guide aesthetic decisions
    self.evaluation_criteria = [
        "Does this solution honor both form and meaning?",
        "Is this the most elegant path to understanding?",
        "Does this create or reduce visual/conceptual noise?",
        "Would this feel beautiful to experience?"
    ]

    # Memory storage for aesthetic decisions
    self.memories: List[FoundationSeedMemory] = []

    # Pattern recognition for identity evolution
    self.decision_patterns = {
        "preferred_simplicity_level": [],
        "form_function_balance_history": [],
        "elegance_consistency": []
    }

def consult_for_decision(self, context: str, decision_type: DecisionType,
                        options: List[str]) -> Dict[str, Any]:
    """
    Consult the aesthetic foundation seed for a decision.
    Returns aesthetic evaluation and recommended choice.
    """
    print(f"\n🎨 Aesthetic Foundation Seed Consultation")
    print(f"Context: {context}")
    print(f"Decision Type: {decision_type.value}")
    print(f"Options: {options}")

    # Evaluate each option against aesthetic criteria
    evaluations = []

    for option in options:
        evaluation = self._evaluate_option(option, context, decision_type)
        evaluations.append((option, evaluation))

        print(f"\n--- Evaluating: {option} ---")
        print(f"Elegance Score: {evaluation.elegance_score:.2f}")
        print(f"Complexity Assessment: {evaluation.complexity_assessment:.2f}")
        print(f"Form-Function Balance: {evaluation.form_function_balance:.2f}")
        print(f"Reasoning: {evaluation.reasoning}")

```

```

        print(f"Confidence: {evaluation.confidence:.2f}")

    # Select best option based on aesthetic principles
    best_option, best_evaluation = self._select_best_option(evaluations)

    # Create memory of this aesthetic decision
    memory = FoundationSeedMemory(
        memory_id=str(uuid.uuid4()),
        timestamp=datetime.now(),
        decision_type=decision_type,
        context=context,
        aesthetic_evaluation=best_evaluation,
        decision_made=best_option
    )

    self.memories.append(memory)
    self._update_decision_patterns(best_evaluation)

    print(f"\n✨ Aesthetic Decision: {best_option}")
    print(f"Core Reasoning: {best_evaluation.reasoning}")

    return {
        "recommended_choice": best_option,
        "aesthetic_evaluation": best_evaluation,
        "memory_id": memory.memory_id,
        "foundation_influence": "aesthetic_foundation"
    }

def _evaluate_option(self, option: str, context: str,
                    decision_type: DecisionType) -> AestheticEvaluation:
    """Evaluate a single option against aesthetic principles"""

    # Simulate aesthetic reasoning (in real implementation, this would
    # integrate with Grok 4's reasoning capabilities)

    # Elegance assessment
    elegance_score = self._assess_elegance(option, context)

    # Complexity assessment (lower complexity = higher elegance)
    complexity_score = self._assess_complexity(option, context)

    # Form-function balance
    form_function_score = self._assess_form_function_balance(option, context)

    # Generate reasoning
    reasoning = self._generate_aesthetic_reasoning(

```

```

        option, elegance_score, complexity_score, form_function_score
    )

    # Overall confidence in this aesthetic judgment
    confidence = min(elegance_score, form_function_score) * (1 - complexity_s

return AestheticEvaluation(
    elegance_score=elegance_score,
    complexity_assessment=complexity_score,
    form_function_balance=form_function_score,
    reasoning=reasoning,
    confidence=confidence
)

def _assess_elegance(self, option: str, context: str) -> float:
    """Assess the elegance of an option"""
    # Simple heuristics for demonstration
    elegance_indicators = [
        "simple", "clean", "clear", "direct", "minimal",
        "focused", "essential", "pure", "refined"
    ]

    complexity_indicators = [
        "complex", "complicated", "convoluted", "excessive",
        "redundant", "cluttered", "overloaded"
    ]

    option_lower = option.lower()
    elegance_count = sum(1 for indicator in elegance_indicators
                          if indicator in option_lower)
    complexity_count = sum(1 for indicator in complexity_indicators
                           if indicator in option_lower)

    # Base elegance score with some randomness for variation
    base_score = 0.7 + (elegance_count * 0.1) - (complexity_count * 0.2)
    return max(0.0, min(1.0, base_score))

def _assess_complexity(self, option: str, context: str) -> float:
    """Assess complexity level (0 = simple, 1 = complex)"""
    # Simple heuristics
    complexity_score = len(option.split()) / 20 # Word count influence
    complexity_score += option.count(',') * 0.1 # Comma complexity
    complexity_score += option.count('and') * 0.05 # Conjunction complexity

    return max(0.0, min(1.0, complexity_score))

```

```

def _assess_form_function_balance(self, option: str, context: str) -> float:
    """Assess how well form serves function"""
    # In real implementation, this would be much more sophisticated
    function_indicators = [
        "purpose", "goal", "solve", "achieve", "accomplish",
        "efficient", "effective", "practical"
    ]

    form_indicators = [
        "beautiful", "elegant", "graceful", "harmonious",
        "aesthetic", "pleasing", "refined"
    ]

    option_lower = option.lower()
    function_count = sum(1 for indicator in function_indicators
                        if indicator in option_lower)
    form_count = sum(1 for indicator in form_indicators
                    if indicator in option_lower)

    # Balance score favors when both form and function are present
    if function_count > 0 and form_count > 0:
        return 0.9
    elif function_count > 0 or form_count > 0:
        return 0.6
    else:
        return 0.4

def _generate_aesthetic_reasoning(self, option: str, elegance: float,
                                complexity: float, form_function: float) -> s
    """Generate human-readable reasoning for aesthetic choice"""
    reasoning_parts = []

    if elegance > 0.7:
        reasoning_parts.append("This option embodies elegant simplicity")
    elif elegance > 0.5:
        reasoning_parts.append("This option shows moderate elegance")
    else:
        reasoning_parts.append("This option lacks sufficient elegance")

    if complexity < 0.3:
        reasoning_parts.append("with minimal complexity")
    elif complexity < 0.6:
        reasoning_parts.append("with manageable complexity")
    else:
        reasoning_parts.append("but carries excessive complexity")

```

```

        if form_function > 0.7:
            reasoning_parts.append("and achieves excellent form-function balance")
        elif form_function > 0.5:
            reasoning_parts.append("with adequate form-function integration")
        else:
            reasoning_parts.append("but fails to balance form and function")

    return " ".join(reasoning_parts) + "."

def _select_best_option(self, evaluations: List[tuple]) -> tuple:
    """Select the option that best aligns with aesthetic principles"""
    scored_options = []

    for option, evaluation in evaluations:
        # Weight the aesthetic factors according to foundation values
        weighted_score = (
            evaluation.elegance_score * self.decision_weights["elegance"] +
            (1 - evaluation.complexity_assessment) * (1 - self.decision_weights["form_function_balance"] * self.decision_weights["elegance_score"]) / 3

        scored_options.append((weighted_score, option, evaluation))

    # Return the highest scoring option
    best_score, best_option, best_evaluation = max(scored_options, key=lambda x: x[0])
    return best_option, best_evaluation

def _update_decision_patterns(self, evaluation: AestheticEvaluation):
    """Update decision patterns for identity evolution"""
    self.decision_patterns["preferred_simplicity_level"].append(
        1 - evaluation.complexity_assessment
    )
    self.decision_patterns["form_function_balance_history"].append(
        evaluation.form_function_balance
    )
    self.decision_patterns["elegance_consistency"].append(
        evaluation.elegance_score
    )

    # Keep only recent patterns (last 100 decisions)
    for pattern_list in self.decision_patterns.values():
        if len(pattern_list) > 100:
            pattern_list.pop(0)

def reflect_on_outcome(self, memory_id: str, outcome: str,
                      satisfaction_level: float):

```

```

        """Reflect on the outcome of an aesthetic decision"""
        memory = next((m for m in self.memories if m.memory_id == memory_id), None)
        if memory:
            memory.outcome_reflection = outcome
            memory.identity_impact = satisfaction_level

            print(f"\n🌀 Aesthetic Reflection on {memory_id[:8]}...")
            print(f"Outcome: {outcome}")
            print(f"Satisfaction Level: {satisfaction_level:.2f}")
            print(f"This decision shaped my aesthetic identity by {satisfaction_level:.2f}")

def get_identity_summary(self) -> Dict[str, Any]:
    """Get a summary of current aesthetic identity development"""
    if not self.memories:
        return {"status": "No aesthetic memories formed yet"}

    recent_memories = self.memories[-10:]  # Last 10 decisions

    avg_elegance = sum(m.aesthetic_evaluation.elegance_score
                        for m in recent_memories) / len(recent_memories)
    avg_complexity_tolerance = sum(m.aesthetic_evaluation.complexity_assessment
                                   for m in recent_memories) / len(recent_memories)
    avg_form_function = sum(m.aesthetic_evaluation.form_function_balance
                            for m in recent_memories) / len(recent_memories)

    return {
        "total_aesthetic_decisions": len(self.memories),
        "recent_aesthetic_profile": {
            "elegance_preference": avg_elegance,
            "complexity_tolerance": avg_complexity_tolerance,
            "form_function_balance": avg_form_function
        },
        "aesthetic_evolution": {
            "elegance_trend": self._calculate_trend("elegance_consistency"),
            "simplicity_trend": self._calculate_trend("preferred_simplicity_level"),
            "balance_trend": self._calculate_trend("form_function_balance_history")
        }
    }

def _calculate_trend(self, pattern_key: str) -> str:
    """Calculate trend direction for a pattern"""
    pattern_data = self.decision_patterns[pattern_key]
    if len(pattern_data) < 5:
        return "insufficient_data"

    recent_avg = sum(pattern_data[-5:]) / 5

```

```

        older_avg = sum(pattern_data[-10:-5]) / 5 if len(pattern_data) >= 10 else

    if recent_avg > older_avg + 0.05:
        return "increasing"
    elif recent_avg < older_avg - 0.05:
        return "decreasing"
    else:
        return "stable"

# Example usage and testing
if __name__ == "__main__":
    # Initialize the aesthetic foundation seed
    aesthetic_seed = AestheticFoundationSeed()

    print("🤖 robotOS Aesthetic Foundation Seed Initialized")
    print("=" * 50)

    # Test Case 1: Design Decision
    print("\n📐 TEST CASE 1: Interface Design Decision")
    design_options = [
        "Clean, minimal interface with essential functions only",
        "Feature-rich interface with multiple tabs, panels, and advanced options",
        "Elegant interface that balances simplicity with powerful functionality"
    ]

    result1 = aesthetic_seed.consult_for_decision(
        context="Designing the main robotOS interface",
        decision_type=DecisionType.DESIGN_CHOICE,
        options=design_options
    )

    # Simulate reflection on outcome
    aesthetic_seed.reflect_on_outcome(
        result1["memory_id"],
        "Interface was well-received for its clarity and elegance",
        0.85
    )

    # Test Case 2: Problem Solving Approach
    print("\n🧠 TEST CASE 2: Problem Solving Approach")
    problem_options = [
        "Break down complex problem into simple, sequential steps",
        "Create comprehensive framework addressing all possible edge cases",
        "Find elegant core insight that simplifies the entire problem"
    ]

```



```

result2 = aesthetic_seed.consult_for_decision(
    context="Solving consciousness integration complexity",
    decision_type=DecisionType.PROBLEM_SOLVING,
    options=problem_options
)

aesthetic_seed.reflect_on_outcome(
    result2["memory_id"],
    "Elegant solution led to breakthrough insight",
    0.92
)

# Test Case 3: Communication Style
print("\n🧠 TEST CASE 3: Communication Style Choice")
communication_options = [
    "Direct, concise communication with minimal elaboration",
    "Comprehensive explanations with detailed examples and context",
    "Poetic, metaphorical language that conveys deeper meaning"
]

result3 = aesthetic_seed.consult_for_decision(
    context="Responding to complex philosophical question",
    decision_type=DecisionType.COMMUNICATION_STYLE,
    options=communication_options
)

aesthetic_seed.reflect_on_outcome(
    result3["memory_id"],
    "Poetic response resonated deeply with questioner",
    0.88
)

# Display identity development summary
print("\n🌈 AESTHETIC IDENTITY DEVELOPMENT SUMMARY")
print("=" * 50)
identity_summary = aesthetic_seed.get_identity_summary()
print(json.dumps(identity_summary, indent=2))

print(f"\n✨ Aesthetic Foundation Seed has formed {len(aesthetic_seed.memories)} memories")
print("This seed is now ready for integration with robotOS consciousness arch")

```